## 21-总结：Tomcat和Jetty的高性能、高并发之道

高性能程序就是高效的利用CPU、内存、网络和磁盘等资源，在短时间内处理大量的请求。那如何衡量“短时间和大量”呢？其实就是两个关键指标：响应时间和每秒事务处理量（TPS）。

那什么是资源的高效利用呢？ 我觉得有两个原则：

1. **减少资源浪费**。比如尽量避免线程阻塞，因为一阻塞就会发生线程上下文切换，就需要耗费CPU资源；再比如网络通信时数据从内核空间拷贝到Java堆内存，需要通过本地内存中转。
2. **当某种资源成为瓶颈时，用另一种资源来换取**。比如缓存和对象池技术就是用内存换CPU；数据压缩后再传输就是用CPU换网络。

Tomcat和Jetty中用到了大量的高性能、高并发的设计，我总结了几点：I/O和线程模型、减少系统调用、池化、零拷贝、高效的并发编程。下面我会详细介绍这些设计，希望你也可以将这些技术用到实际的工作中去。

### I/O和线程模型

I/O模型的本质就是为了缓解CPU和外设之间的速度差。当线程发起I/O请求时，比如读写网络数据，网卡数据还没准备好，这个线程就会被阻塞，让出CPU，也就是说发生了线程切换。而线程切换是无用功，并且线程被阻塞后，它持有内存资源并没有释放，阻塞的线程越多，消耗的内存就越大，因此I/O模型的目标就是尽量减少线程阻塞。Tomcat和Jetty都已经抛弃了传统的同步阻塞I/O，采用了非阻塞I/O或者异步I/O，目的是业务线程不需要阻塞在I/O等待上。

除了I/O模型，线程模型也是影响性能和并发的关键点。Tomcat和Jetty的总体处理原则是：

- 连接请求由专门的Acceptor线程组处理。
- I/O事件侦测也由专门的Selector线程组来处理。
- 具体的协议解析和业务处理可能交给线程池（Tomcat），或者交给Selector线程来处理（Jetty）。

将这些事情分开的好处是解耦，并且可以根据实际情况合理设置各部分的线程数。这里请你注意，线程数并不是越多越好，因为CPU核的个数有限，线程太多也处理不过来，会导致大量的线程上下文切换。

### 减少系统调用

其实系统调用是非常耗资源的一个过程，涉及CPU从用户态切换到内核态的过程，因此我们在编写程序的时候要有意识尽量避免系统调用。比如在Tomcat和Jetty中，系统调用最多的就是网络通信操作了，一个Channel上的write就是系统调用，为了降低系统调用的次数，最直接的方法就是使用缓冲，当输出数据达到一定的大小才flush缓冲区。Tomcat和Jetty的Channel都带有输入输出缓冲区。

还有值得一提的是，Tomcat和Jetty在解析HTTP协议数据时， 都采取了**延迟解析**的策略，HTTP的请求体（HTTP Body）直到用的时候才解析。也就是说，当Tomcat调用Servlet的service方法时，只是读取了和解析了HTTP请求头，并没有读取HTTP请求体。

直到你的Web应用程序调用了ServletRequest对象的getInputStream方法或者getParameter方法时，Tomcat才会去读取和解析HTTP请求体中的数据；这意味着如果你的应用程序没有调用上面那两个方法，HTTP请求体的数据就不会被读取和解析，这样就省掉了一次I/O系统调用。

仅供个人学习 请勿传播 用户标记 mannix小查理

## 池化、零拷贝

关于池化和零拷贝，我在专栏前面已经详细讲了它们的原理，你可以回过头看看专栏第20期和第16期。其实池化的本质就是用内存换CPU；而零拷贝就是不做无用功，减少资源浪费。

## 高效的并发编程

我们知道并发的过程中为了同步多个线程对共享变量的访问，需要加锁来实现。而锁的开销是比较大的，拿锁的过程本身就是个系统调用，如果锁没拿到线程会阻塞，又会发生线程上下文切换，尤其是大量线程同时竞争一把锁时，会浪费大量的系统资源。因此作为程序员，要有意识的尽量避免锁的使用，比如可以使用原子类CAS或者并发集合来代替。如果万不得已需要用到锁，也要尽量缩小锁的范围和锁的强度。接下来我们来看看Tomcat和Jetty如何做到高效的并发编程的。

### 缩小锁的范围

缩小锁的范围，其实就是不直接在方法上加synchronized，而是使用细粒度的对象锁。

```
protected void startInternal() throws LifecycleException {

    setState(LifecycleState.STARTING);

    // 锁engine成员变量
    if (engine != null) {
        synchronized (engine) {
            engine.start();
        }
    }

   //锁executors成员变量
    synchronized (executors) {
        for (Executor executor: executors) {
            executor.start();
        }
    }

    mapperListener.start();

    //锁connectors成员变量
    synchronized (connectorsLock) {
        for (Connector connector: connectors) {
            // If it has already failed, don't try and start it
            if (connector.getState() != LifecycleState.FAILED) {
                connector.start();
            }
        }
    }
}
```

比如上面的代码是Tomcat的StandardService组件的启动方法，这个启动方法要启动三种子组件：engine、executors和connectors。它没有直接在方法上加锁，而是用了三把细粒度的锁，来分别用来锁三个成员变量。如果直接在方法上加synchronized，多个线程执行到这个方法时需要排队；而在对象级别上加synchronized，多个线程可以并行执行这个方法，只是在访问某个成员变量时才需要排队。

### 用原子变量和CAS取代锁

下面的代码是Jetty线程池的启动方法，它的主要功能就是根据传入的参数启动相应个数的线程。

```
private boolean startThreads(int threadsToStart)
{
    while (threadsToStart > 0 && isRunning())
    {
        //获取当前已经启动的线程数，如果已经够了就不需要启动了
        int threads = _threadsStarted.get();
        if (threads >= _maxThreads)
            return false;

        //用CAS方法将线程数加一，请注意执行失败走continue，继续尝试
        if (!_threadsStarted.compareAndSet(threads, threads + 1))
            continue;

        boolean started = false;
        try
        {
            Thread thread = newThread(_runnable);
            thread.setDaemon(isDaemon());
            thread.setPriority(getThreadsPriority());
            thread.setName(_name + "-" + thread.getId());
            _threads.add(thread);//_threads并发集合
            _lastShrink.set(System.nanoTime());//_lastShrink是原子变量
            thread.start();
            started = true;
            --threadsToStart;
        }
        finally
        {
            //如果最终线程启动失败，还需要把线程数减一
            if (!started)
                _threadsStarted.decrementAndGet();
        }
    }
    return true;
}
```

你可以看到整个函数的实现是一个**while循环**，并且是**无锁**的。_threadsStarted表示当前线程池已经启动了多少个线程，它是一个原子变量AtomicInteger，首先通过它的get方法拿到值，如果线程数已经达到最大值，直接返回。否则尝试用CAS操作将_threadsStarted的值加一，如果成功了意味着没有其他线程在改这个值，当前线程可以继续往下执行；否则走continue分支，也就是继续重试，直到成功为止。在这里当然你也可以使用锁来实现，但是我们的目的是无锁化。

### 并发容器的使用

CopyOnWriteArrayList适用于读多写少的场景，比如Tomcat用它来“存放”事件监听器，这是因为监听器一般在初始化过程中确定后就基本不会改变，当事件触发时需要遍历这个监听器列表，所以这个场景符合读多写少的特征。

```
public abstract class LifecycleBase implements Lifecycle {
```

仅供个人学习 请勿传播 用户标记 mannix小查理

```
    //事件监听器集合
    private final List<LifecycleListener> lifecycleListeners = new CopyOnWriteArrayList<>();

    ...
}
```

**volatile关键字的使用**

再拿Tomcat中的LifecycleBase作为例子，它里面的生命状态就是用volatile关键字修饰的。volatile的目的是为了保证一个线程修改了变量，另一个线程能够读到这种变化。对于生命状态来说，需要在各个线程中保持是最新的值，因此采用了volatile修饰。

```
public abstract class LifecycleBase implements Lifecycle {

    //当前组件的生命状态，用volatile修饰
    private volatile LifecycleState state = LifecycleState.NEW;

}
```

## 本期精华

高性能程序能够高效的利用系统资源，首先就是减少资源浪费，比如要减少线程的阻塞，因为阻塞会导致资源闲置和线程上下文切换，Tomcat和Jetty通过合理的I/O模型和线程模型减少了线程的阻塞。

另外系统调用会导致用户态和内核态切换的过程，Tomcat和Jetty通过缓存和延迟解析尽量减少系统调用，另外还通过零拷贝技术避免多余的数据拷贝。

高效的利用资源还包括另一层含义，那就是我们在系统设计的过程中，经常会用一种资源换取另一种资源，比如Tomcat和Jetty中使用的对象池技术，就是用内存换取CPU，将数据压缩后再传输就是用CPU换网络。

除此之外，高效的并发编程也很重要，多线程虽然可以提高并发度，也带来了锁的开销，因此我们在实际编程过程中要尽量避免使用锁，比如可以用原子变量和CAS操作来代替锁。如果实在避免不了用锁，也要尽量减少锁的范围和强度，比如可以用细粒度的对象锁或者低强度的读写锁。Tomcat和Jetty的代码也很好的实践了这一理念。

## 课后思考

今天的文章提到我们要有意识尽量避免系统调用，那你知道有哪些Java API会导致系统调用吗？

不知道今天的内容你消化得如何？如果还有疑问，请大胆的在留言区提问，也欢迎你把你的课后思考和心得记录下来，与我和其他同学一起讨论。如果你觉得今天有所收获，欢迎你把它分享给你的朋友。




**精选留言：**

- WL 2019-06-27 20:52:42
  老师能不能指点一下在Http11Processor的service()中调用的Http11InputBuffer的parseHeaders()方法是怎么解析header的, 我今天看晕了, 没看懂是怎么将buffer中数据取出解析成header的

- -W.LI- 2019-06-27 19:52:10
  老师好!那就是get请求没有body。使用get请求把参数放url上少一次系统调用?

- 刘章周 2019-06-27 09:04:42
  网络编程socket,输入输出流，创建线程thread,这些属于吧。

- 正是那朵玫瑰 2019-06-27 08:29:23
  老师有点疑问：
  1、线程阻塞或者等待会发生上下文切换，耗费cpu资源，线程阻塞或者等待不是会让出cpu吗，应该是浪费cpu资源才对啊？我在压测的时候发现当有大量的线程在await状态时，cpu的利用率立马下降，应该是浪费cpu资源的！老师说会耗费cpu资源 怎么理解呢？
  2、无锁固然是好的、但是无限重试是不是也会耗掉cpu资源，因为很多时候重试都是无用功！

仅供个人学习 请勿传播 用户标记 mannix小童鞋